

# NutriLens — Documento funcional y alcance del MVP

## 1. Resumen ejecutivo

**NutriLens** es una aplicación inteligente para analizar etiquetas de alimentos a partir de imágenes o PDFs. El usuario sube una foto del producto, etiqueta nutricional o ingredientes, y el sistema extrae información estructurada, detecta posibles alérgenos, identifica sellos o advertencias, clasifica el riesgo del producto y genera una explicación simple.

La propuesta busca resolver un problema cotidiano: muchas personas no entienden rápidamente si un alimento es compatible con sus restricciones, preferencias o necesidades. NutriLens no reemplaza a un profesional de nutrición ni garantiza seguridad médica absoluta; funciona como un **asistente informativo** basado en la información visible de la etiqueta.

---

## 2. Objetivo general

Diseñar y desarrollar una aplicación inteligente capaz de procesar etiquetas alimentarias mediante IA multimodal, extraer información relevante y ayudar al usuario a interpretar productos de forma clara, rápida y estructurada.

**IA multimodal** significa que el modelo puede analizar más de un tipo de entrada, por ejemplo imagen y texto.

---

## 3. Objetivos específicos

- Permitir la carga de imágenes o PDFs de etiquetas alimentarias.
  - Detectar si el archivo corresponde a un producto alimenticio válido.
  - Extraer ingredientes, alérgenos, sellos, categoría del producto y datos nutricionales visibles.
  - Normalizar la información en formato **JSON**, que es un formato de datos basado en pares clave-valor.
  - Clasificar el producto según riesgo bajo, medio o alto.
  - Mostrar una explicación simple y entendible para el usuario.
  - Guardar productos analizados en una base propia.
  - Permitir consultas sobre productos guardados mediante un chat con recuperación de información.
  - Mostrar el flujo interno del análisis para hacer visible cómo trabaja el sistema.
- 

## 4. Problema a resolver

Las etiquetas alimentarias suelen tener información técnica, extensa o difícil de interpretar rápidamente. Un usuario puede no saber si un producto contiene gluten, lactosa, ingredientes de origen animal, exceso de azúcar o componentes problemáticos para sus preferencias.

Ejemplos de problemas reales:

- Una persona celíaca quiere saber si un producto parece compatible.
  - Una persona con intolerancia a la lactosa quiere detectar ingredientes derivados de leche.
  - Un usuario quiere comparar galletitas y elegir la opción con mejor perfil nutricional.
  - Un consumidor quiere entender sellos o advertencias sin leer toda la etiqueta.
  - Un grupo quiere consultar productos ya analizados y recibir recomendaciones.
- 

## 5. Usuarios objetivo

### Usuario general

Persona que quiere entender rápidamente una etiqueta alimentaria.

### Usuario con restricciones alimentarias

Persona que quiere identificar posibles incompatibilidades como gluten, lactosa, ingredientes animales o alérgenos.

### Usuario comparador

Persona que quiere consultar productos guardados para elegir entre varias opciones.

---

## 6. Alcance del MVP

**MVP** significa **Producto Mínimo Viable**: una primera versión funcional, acotada y demostrable que resuelve el caso de uso principal.

El MVP de NutriLens incluirá:

1. Carga de imagen o PDF de etiqueta alimentaria.
  2. Validación básica del archivo.
  3. Extracción de datos mediante modelo de IA multimodal.
  4. Normalización de salida en JSON.
  5. Clasificación de alérgenos y restricciones.
  6. Cálculo de riesgo mediante reglas propias.
  7. Visualización clara del análisis.
  8. Guardado del producto procesado.
  9. Historial de productos analizados.
  10. Chat simple para consultar productos guardados.
- 

## 7. Fuera del alcance del MVP

Para mantener el proyecto viable, el MVP no incluirá inicialmente:

- Diagnóstico médico o nutricional.

- Recomendaciones clínicas personalizadas.
- Escaneo obligatorio de código de barras.
- Integración completa obligatoria con Open Food Facts.
- App móvil nativa.
- Registro avanzado de usuarios.
- Planes nutricionales.
- Comparador complejo de productos.
- Audio generado.
- Generación sintética de etiquetas.
- Base masiva de alimentos reales.

Estos puntos pueden considerarse para una versión final o como funcionalidades bonus.

---

## 8. Funcionalidades principales

### 8.1. Carga de producto

El usuario puede subir:

- Foto del frente del producto.
- Foto de ingredientes.
- Foto de tabla nutricional.
- PDF con información del producto.

El sistema muestra estados claros:

- Archivo cargado.
- Procesando.
- Análisis completado.
- Error de archivo no soportado.

### 8.2. Validación de entrada

El sistema valida si la imagen o PDF corresponde a una etiqueta alimentaria.

Si no es válida, responde con una estructura de error:

```
{
  "error": "image_not_supported",
  "reason": "La imagen no parece corresponder a una etiqueta alimentaria."
}
```

### 8.3. Extracción estructurada

La IA extrae información visible y la transforma en JSON.

Ejemplo:

```
{
  "producto": "Galletitas Choco Crunch",
  "categoria": "Galletitas dulces",
  "ingredientes_detectados": [
    "harina de trigo",
    "azúcar",
    "aceite vegetal",
    "cacao",
    "leche en polvo"
  ],
  "alergenos": ["gluten", "leche"],
  "sellos": ["exceso en azúcares", "exceso en grasas saturadas"],
  "apto_vegano": false,
  "apto_celiaco": false,
  "apto_sin_lactosa": false,
  "riesgo": "alto",
  "confidence": 0.94
}
```

#### 8.4. Clasificación de riesgo

El sistema asigna riesgo bajo, medio o alto.

Reglas iniciales:

- Si contiene gluten, no se considera apto celíaco.
- Si contiene leche, lactosa, suero o derivados lácteos, no se considera apto sin lactosa.
- Si contiene ingredientes de origen animal, no se considera vegano.
- Si tiene dos o más sellos, aumenta el riesgo.
- Si la confianza de extracción es baja, se muestra advertencia.

#### 8.5. Explicación simple

La app traduce la salida técnica a lenguaje claro.

Ejemplo:

Este producto no es recomendable para personas con celiaquía o intolerancia a la lactosa.  
Se detectan gluten, leche y alto contenido de azúcar.

#### 8.6. Historial de productos

Cada análisis queda guardado con:

- Nombre del producto.
- Categoría.
- Ingredientes.
- Alérgenos.
- Sellos.
- Riesgo.

- Fecha de análisis.
- Imagen asociada.
- Resultado JSON.

## 8.7. Chat con recuperación de información

El usuario puede preguntar sobre productos ya cargados.

Ejemplos:

- “Dame galletitas con mejor perfil nutricional.”
- “Mostrame productos aptos para celíacos.”
- “¿Qué productos escaneados tienen leche?”
- “Compareme estas dos opciones.”

El sistema recupera productos relevantes y genera una respuesta usando esos datos.

Esto es una versión simple de **RAG**, que significa **Retrieval-Augmented Generation**. RAG consiste en buscar información guardada y usarla como contexto para que el modelo genere una respuesta fundamentada.

---

## 9. Flujo principal del usuario

### Flujo 1: analizar producto

1. El usuario ingresa a NutriLens.
2. Selecciona “Analizar producto”.
3. Sube una foto o PDF de la etiqueta.
4. La app muestra estado “Procesando imagen”.
5. El backend envía el archivo al modelo multimodal.
6. La IA devuelve datos estructurados.
7. El backend normaliza el JSON.
8. Se aplican reglas propias de clasificación.
9. Se guarda el producto.
10. La UI muestra resumen, riesgo, alérgenos, ingredientes y explicación.

### Flujo 2: entrada inválida

1. El usuario sube una imagen que no es una etiqueta alimentaria.
2. El sistema detecta que no puede procesarla.
3. Devuelve error estructurado.
4. La UI muestra el motivo y permite cargar otro archivo.

### Flujo 3: consulta por chat

1. El usuario pregunta: “Dame galletitas saludables”.
2. El sistema interpreta la intención.
3. Busca productos de categoría “galletitas” en la base.
4. Ordena candidatos por menor riesgo, menos sellos y mejor perfil.
5. Envía los productos relevantes al modelo.

6. El modelo genera una respuesta explicada.
7. La UI muestra recomendaciones y advertencias.

#### Flujo 4: historial

1. El usuario entra a "Historial".
2. Ve productos analizados anteriormente.
3. Filtra por categoría, riesgo o alérgeno.
4. Puede abrir el detalle de un producto.
5. Puede hacer preguntas sobre ese historial.

---

## 10. Requerimientos funcionales

| Código | Requerimiento                                                                |
|--------|------------------------------------------------------------------------------|
| RF-01  | El sistema debe permitir subir imágenes de etiquetas alimentarias.           |
| RF-02  | El sistema debe permitir subir PDFs.                                         |
| RF-03  | El sistema debe validar si el archivo corresponde a un producto alimenticio. |
| RF-04  | El sistema debe extraer ingredientes visibles.                               |
| RF-05  | El sistema debe detectar alérgenos principales.                              |
| RF-06  | El sistema debe detectar sellos o advertencias visibles.                     |
| RF-07  | El sistema debe clasificar aptitud vegana, celíaca y sin lactosa.            |
| RF-08  | El sistema debe calcular riesgo bajo, medio o alto.                          |
| RF-09  | El sistema debe mostrar explicación simple del análisis.                     |
| RF-10  | El sistema debe guardar productos analizados.                                |
| RF-11  | El sistema debe mostrar historial de productos.                              |
| RF-12  | El sistema debe permitir consultas en chat sobre productos guardados.        |
| RF-13  | El sistema debe devolver errores estructurados ante imágenes inválidas.      |
| RF-14  | El sistema debe mostrar el JSON extraído.                                    |
| RF-15  | El sistema debe mostrar el pipeline del análisis.                            |

---

## 11. Requerimientos no funcionales

| Código | Requerimiento                                          |
|--------|--------------------------------------------------------|
| RNF-01 | La interfaz debe ser clara, simple y responsive.       |
| RNF-02 | El análisis debe mostrar estados de carga y error.     |
| RNF-03 | El sistema debe evitar afirmaciones médicas absolutas. |

| Código | Requerimiento                                                              |
|--------|----------------------------------------------------------------------------|
| RNF-04 | El sistema debe aclarar que el análisis depende de la información visible. |
| RNF-05 | La salida del modelo debe validarse antes de guardarse.                    |
| RNF-06 | El JSON debe tener una estructura consistente.                             |
| RNF-07 | El sistema debe permitir datos mockeados para demo estable.                |
| RNF-08 | El tiempo de análisis debería mantenerse razonable para una presentación.  |

## 12. Datos iniciales para el MVP

Conviene precargar entre 20 y 40 productos para asegurar una demo estable.

Categorías sugeridas:

- Galletitas.
- Cereales.
- Snacks.
- Lácteos.
- Bebidas.
- Productos sin TACC.
- Productos veganos.

Cada producto debería tener:

```
{
  "id": "prod_001",
  "nombre": "Galletitas Choco Crunch",
  "categoria": "galletitas",
  "ingredientes": ["harina de trigo", "azúcar", "leche en polvo"],
  "alergenos": ["gluten", "leche"],
  "sellos": ["exceso en azúcares"],
  "apto_vegano": false,
  "apto_celiaco": false,
  "apto_sin_lactosa": false,
  "riesgo": "alto"
}
```

## 13. Integración con Open Food Facts

Open Food Facts puede integrarse como fuente externa opcional para enriquecer productos.

Flujo propuesto para versión posterior al MVP:

1. Usuario ingresa o escanea código de barras.

2. La app consulta Open Food Facts.
3. Obtiene ingredientes, datos nutricionales y metadata disponible.
4. La IA normaliza esa información al formato propio.
5. Se guarda en la base local.
6. El producto queda disponible para consultas RAG.

Para el MVP, Open Food Facts no debería ser obligatorio. La app debe funcionar con carga de imagen/PDF y base propia.

---

## 14. Arquitectura propuesta

### Frontend

- Next.js.
- TypeScript.
- Componentes para upload, resultado, historial y chat.

### Backend

- API Routes de Next.js o backend Node.js.
- Endpoint para analizar imagen/PDF.
- Endpoint para historial.
- Endpoint para chat/RAG.

### IA

- Modelo multimodal para analizar imagen/PDF.
- Modelo de lenguaje para explicación y chat.

LLM significa **Large Language Model**, modelo de lenguaje grande capaz de interpretar y generar texto.

### Base de datos

- SQLite, PostgreSQL o archivo JSON para MVP.
- Tabla de productos analizados.
- Tabla opcional de consultas realizadas.

### RAG MVP

- Búsqueda por filtros y palabras clave.
  - Envío de productos recuperados al modelo.
  - Generación de respuesta usando solo productos disponibles.
- 

## 15. Pipeline del análisis

1. **Ingesta de archivo**: recibe imagen o PDF.
2. **Validación**: verifica si parece una etiqueta alimentaria.
3. **OCR / lectura visual**: extrae texto visible.
4. **Parsing**: convierte texto desordenado en campos estructurados.



5. **Detección de alérgenos:** identifica gluten, leche, frutos secos u otros.
6. **Clasificación de aptitud:** evalúa vegano, celíaco y sin lactosa.
7. **Cálculo de riesgo:** aplica reglas propias.
8. **Generación de explicación:** redacta una salida simple.
9. **Persistencia:** guarda el producto.
10. **Consulta RAG:** permite preguntas futuras sobre productos guardados.

**OCR** significa **Optical Character Recognition**, reconocimiento óptico de caracteres. Es la técnica usada para convertir texto dentro de una imagen en texto procesable.

---

## 16. Pantallas principales

### Pantalla 1: inicio / upload

Elementos:

- Botón para subir imagen o PDF.
- Instrucciones de uso.
- Ejemplos de imágenes válidas.
- Acceso a historial.

### Pantalla 2: análisis procesado

Elementos:

- Imagen del producto.
- Riesgo general.
- Aptitud vegana/celíaca/sin lactosa.
- Sellos detectados.
- Ingredientes.
- Alérgenos.
- Explicación simple.
- JSON extraído.
- Pipeline del agente.

### Pantalla 3: historial

Elementos:

- Lista de productos analizados.
- Filtros por categoría, riesgo y alérgenos.
- Acceso al detalle.

### Pantalla 4: chat/RAG

Elementos:

- Input de pregunta.
- Productos recuperados.
- Respuesta explicada.

- Advertencias si no hay datos suficientes.
- 

## 17. Casos de uso principales

### Caso de uso 1: analizar galletitas

**Actor:** Usuario general.

**Entrada:** Foto de etiqueta de galletitas.

**Salida:** Riesgo alto, gluten, leche, exceso en azúcares, explicación simple.

### Caso de uso 2: buscar productos aptos para celíacos

**Actor:** Usuario con restricción alimentaria.

**Entrada:** "Mostrame productos aptos para celíacos".

**Salida:** Lista de productos guardados sin gluten detectado.

### Caso de uso 3: imagen inválida

**Actor:** Usuario general.

**Entrada:** Foto que no corresponde a alimento.

**Salida:** Error estructurado con razón.

### Caso de uso 4: recomendar mejor opción dentro de una categoría

**Actor:** Usuario comparador.

**Entrada:** "Dame galletitas con mejor perfil nutricional".

**Salida:** Ranking de productos guardados con explicación.

---

## 18. Criterios de aceptación del MVP

- El usuario puede subir una imagen o PDF.
  - El sistema puede devolver un JSON estructurado.
  - El sistema detecta al menos ingredientes, alérgenos y riesgo.
  - La UI muestra el análisis de forma entendible.
  - El sistema guarda el producto analizado.
  - El historial muestra productos previos.
  - El chat responde usando productos guardados.
  - Si la imagen no es válida, devuelve error controlado.
  - El sistema no afirma seguridad médica absoluta.
- 

## 19. Riesgos y mitigaciones

| Riesgo         | Impacto               | Mitigación                               |
|----------------|-----------------------|------------------------------------------|
| Imagen borrosa | Extracción incorrecta | Mostrar confidence y pedir nueva imagen. |

---

| Riesgo                           | Impacto | Mitigación                                                |
|----------------------------------|---------|-----------------------------------------------------------|
| IA inventa datos                 | Alto    | Validar schema y usar reglas propias.                     |
| Producto no está en base externa | Medio   | Permitir análisis por imagen y base propia.               |
| Respuestas médicas absolutas     | Alto    | Usar disclaimers y lenguaje informativo.                  |
| Poco tiempo de desarrollo        | Alto    | Recortar a upload, JSON, riesgo, historial y chat simple. |

## 20. División de tareas para 4 personas

| Persona   | Responsabilidad                                              |
|-----------|--------------------------------------------------------------|
| Persona 1 | Frontend: upload, resultado, historial y UI general.         |
| Persona 2 | Backend: endpoints, base de datos y persistencia.            |
| Persona 3 | IA: prompts, schema JSON, extracción y explicación.          |
| Persona 4 | RAG, datos mockeados, testing, documentación y presentación. |

## 21. Roadmap sugerido

### Semana 1

- Definir schema JSON.
- Armar UI base.
- Implementar upload.
- Crear endpoint de análisis.
- Probar prompt con 5 productos.
- Guardar resultados en base.

### Semana 2

- Agregar historial.
- Agregar chat/RAG básico.
- Cargar productos mockeados.
- Mejorar clasificación de riesgo.
- Pulir UI.
- Crear casos de prueba.
- Preparar presentación.

### Versión final / bonus

- Integrar Open Food Facts.
- Agregar escaneo de código de barras.

- Agregar búsqueda vectorial real.
  - Agregar comparador de productos.
  - Agregar audio resumen.
  - Agregar generación de etiqueta simplificada.
- 

## 22. Métricas de éxito

- Porcentaje de análisis completados correctamente.
  - Cantidad de campos extraídos por producto.
  - Precisión observada en detección de alérgenos sobre casos de prueba.
  - Tiempo promedio de análisis.
  - Cantidad de productos guardados.
  - Cantidad de consultas respondidas usando historial.
  - Cantidad de errores controlados.
- 

## 23. Frase de presentación

NutriLens es una app inteligente que analiza etiquetas alimentarias desde imágenes o PDFs, extrae información estructurada con IA multimodal, clasifica riesgos mediante reglas propias y permite consultar productos guardados usando recuperación de información.

---

## 24. Conclusión

NutriLens es un MVP viable, demostrable y alineado con una aplicación inteligente real. Su valor está en combinar procesamiento de imagen, extracción estructurada, reglas de clasificación, explicación en lenguaje simple y consultas sobre productos guardados.

El alcance propuesto evita depender de una base externa completa o de recomendaciones médicas. En cambio, se enfoca en una experiencia concreta: ayudar al usuario a entender mejor la información visible de una etiqueta alimentaria.